

A Dynamic Plugin-Driven Micro-frontend Shell for Configurable WebXR Experiences

Duong Thai Bao¹, Nguyen Thanh Luan¹, Tran The Trung^{1*}

¹*Faculty of Information Technology, Industrial University of Ho Chi Minh City*
**tranhettrung@iuh.edu.vn*

Abstract—This paper introduces a novel content-defined runtime micro-frontend architecture specifically designed for immersive WebXR applications. Unlike traditional micro-frontend models—typically based on route-level composition or build-time bundling—our system dynamically loads, mounts and updates UI modules (“plugins”) in real time based solely on backend-delivered content. The architecture integrates three core innovations: (1) a runtime plugin orchestrator supporting dynamic module loading and unified lifecycle management; (2) a plugin lifecycle contract enabling heterogeneous frontend modules to operate consistently; and (3) an overlay-driven micro-frontend shell offering seamless immersive interaction independent of the VR scene. We implement the architecture in a large-scale WebXR campus exploration system featuring 22 interactive locations and multiple plugin types including gallery, iframe and notification modules. Experimental evaluation demonstrates significant improvements in extensibility, reconfigurability and development agility compared to monolithic VR UI frameworks. Results confirm that the proposed architecture reduces deployment overhead, enhances modularity and enables non-technical operators to configure VR experiences without code changes. This work contributes a new perspective on micro-frontend design for immersive systems and opens pathways toward future AI-driven adaptive VR interfaces.

Keywords. WebXR, Micro-frontend, Runtime Micro-frontend, Plugin-based UI, Dynamic Plugin Loading, Overlay UI, VR Interface, Content-defined UI, A-Frame, Node.js.

Tóm tắt. Nghiên cứu này đề xuất một kiến trúc micro-frontend thời gian thực dựa trên nội dung dành cho các ứng dụng WebXR nhập vai. Khác với các mô hình micro-frontend truyền thống – vốn chủ yếu dựa trên phân tách theo tuyến (route-based) hoặc biên dịch (build-time composition) – hệ thống của chúng tôi cho phép tải, gắn, cập nhật và gỡ bỏ các mô-đun giao diện (“plugin”) hoàn toàn ở thời gian thực dựa trên nội dung được trả về từ backend. Kiến trúc gồm ba điểm đổi mới chính: (1) bộ điều phối plugin thời gian thực (runtime orchestrator) quản lý import module động và vòng đời plugin; (2) chuẩn vòng đời plugin thống nhất, giúp các mô-đun giao diện khác nhau vận hành đồng nhất; và (3) lớp vỏ overlay hoạt động như một micro-frontend shell, tách biệt khỏi cảnh VR và điều phối hành vi giao diện. Kiến trúc đã được triển khai trong hệ thống tham quan WebXR quy mô lớn với 22 địa điểm, sử dụng các plugin như gallery, iframe và thông báo. Kết quả đánh giá cho thấy hệ thống cải thiện đáng kể khả năng mở rộng, rút ngắn chu kỳ phát triển và nâng cao tính linh hoạt so với kiến trúc VR nguyên khối truyền thống. Công trình này mở ra một hướng tiếp cận mới cho micro-frontend

trong môi trường nhập vai và đặt nền tảng cho các hệ thống VR thích ứng theo dữ liệu trong tương lai.

Từ khóa: WebXR, Micro-frontend, Micro-frontend thời gian thực, Giao diện plugin, Tải plugin động, Overlay UI, Giao diện VR, UI theo nội dung, A-Frame, Node.js.

I. GIỚI THIỆU

A. Bối cảnh

Trong nghiên cứu này, chúng tôi sử dụng thuật ngữ WebXR thay cho WebVR, vốn đã bị ngừng hỗ trợ từ năm 2020.

Sự phát triển của WebXR đã mở ra khả năng cung cấp trải nghiệm 3D trực tiếp trên trình duyệt mà không cần phần mềm cài đặt [1], [2]. Nhiều nghiên cứu khẳng định VR đang trở thành xu hướng trong giáo dục và mô phỏng [3]–[5].

Trong khi đó, micro-frontend nổi lên như giải pháp mở rộng quy mô ứng dụng web lớn, phân chia hệ thống thành các module giao diện độc lập [6], [7]. Tuy nhiên, các mô hình micro-frontend truyền thống thường dựa vào điều hướng tuyến (route-based) hoặc ghép nối ở thời điểm biên dịch (build-time) [8], vốn không phù hợp với môi trường VR – nơi UI được kích hoạt theo tương tác trong không gian 3D, không theo tuyến đường URL.

Đồng thời, giao diện WebXR truyền thống thường là UI nguyên khối, gây khó mở rộng và phụ thuộc chặt vào logic cảnh VR [9].

Những hạn chế trên cho thấy nhu cầu cấp thiết về một kiến trúc micro-frontend phù hợp cho WebXR, hoạt động hoàn toàn ở runtime và không phụ thuộc vào cấu trúc scene 3D.

B. Đóng góp chính

Đề xuất một mô hình micro-frontend vận hành ở thời gian thực, trong đó các plugin được tải bằng dynamic import dựa trên nội dung do backend cung cấp

Định nghĩa một plugin lifecycle contract chuẩn hoá một cơ chế vận hành nhất quán.

Một overlay-driven micro-frontend shell tách khỏi scene 3D.

Cung cấp một case-study triển khai trong hệ thống WebXR campus gồm 22 điểm tương tác.

II. CÁC NGHIÊN CỨU LIÊN QUAN

A. Các mô hình micro-frontend hiện có và hạn chế trong WebXR

Micro-frontends được đề cập đầu tiên bởi Luca Mezzalana [6] và sau đó được mở rộng qua các công trình về mô hình phân tách giao diện [7], [8].

Các mô hình micro-frontend phổ biến hiện nay chủ yếu xoay quanh ba hướng tiếp cận.

1. Route-based micro-frontend: mỗi route ứng với một ứng dụng con độc lập, được mount/unmount dựa trên điều hướng URL. Cách tiếp cận này phù hợp với ứng dụng web 2D dạng dashboard, nhưng trong môi trường WebXR các tương tác lại diễn ra trong không gian 3D và không gắn trực tiếp với route, nên rất khó áp dụng mô hình này cho overlay UI được kích hoạt theo điểm tương tác trong scene [6].

2. Build-time composition: các module giao diện được liên kết với nhau ở thời điểm build, giúp chia nhỏ codebase nhưng

vẫn đòi hỏi biên dịch lại khi thay đổi hành vi UI. Điều này làm hạn chế khả năng tái cấu hình trải nghiệm VR theo thời gian thực, đặc biệt khi hệ thống cần thử nghiệm nhanh nhiều kịch bản nội dung khác nhau [8].

3. Web Components: cho phép đóng gói và tái sử dụng thành phần giao diện, nhưng vẫn thiếu một cơ chế orchestration runtime thống nhất để quyết định thành phần nào sẽ được tải, mount và update dựa trên dữ liệu nội dung từ backend. Các nghiên cứu hiện tại chưa đề xuất một plugin shell và lifecycle contract dành riêng cho UI overlay trong WebXR [10].

Phương pháp	Cơ chế chính	Điểm mạnh	Hạn chế trong WebXR
Route-based MF	Mỗi route là 1 app con	Dễ tách team, rõ ràng theo URL	UI VR không theo route, khó kích hoạt theo điểm trong scene
Build-time composition	Ghép module lúc build	Giảm kích thước bundle, chia nhỏ	Mỗi thay đổi trải nghiệm phải build lại, không runtime
Web Component s MF	Custom Elements, Shadow DOM	Đóng gói, tái sử dụng tốt	Thiếu orchestrator và lifecycle runtime cho VR overlay

Từ bảng so sánh có thể thấy các phương pháp hiện tại chưa đáp ứng đồng thời yêu cầu: (i) runtime composition, (ii) tách biệt khỏi VR scene và (iii) điều khiển hoàn toàn bởi cấu trúc nội dung.

B. Plugin architecture

Plugin system đã được nghiên cứu rộng rãi trong lĩnh vực IDE, CMS và game engine [11].

Tuy nhiên, đa số:

- Phụ thuộc vào môi trường host tĩnh
- Không hỗ trợ import module động bằng dữ liệu
- Không có chuẩn vòng đời thống nhất trong bối cảnh VR

C. WebXR và UI overlay

Nghiên cứu của Mozilla và cộng đồng WebXR cho thấy overlay UI là yếu tố quan trọng của VR trên web, nhưng chưa có cơ chế chính thức để phân tách UI thành module độc lập [1], [2], [12].

Các nghiên cứu VR trong giáo dục (Ceulemans, Cardona, Zhao...) [3]–[5] chỉ tập trung vào trải nghiệm người dùng, không đề cập tới kiến trúc mở rộng UI.

III. KHOẢNG TRỐNG NGHIÊN CỨU

Chưa có công trình nào giải quyết đồng thời:

Thành phần	Tình trạng nghiên cứu hiện tại	Khoảng trống
Micro-frontend Runtime	Route-based, Build-time	Không hỗ trợ runtime load cho VR
Plugin Architecture	Static Host Plugins	Thiếu lifecycle và dynamic import

WebXR Overlay	UI rời rạc	Không modular, không plugin shell
Configurable Experience	Manual configuration	Không có Content-defined Micro-Frontend

Chính vì vậy, trong nghiên cứu này nhóm nghiên cứu đề xuất một kiến trúc Plugin-Driven Micro-frontend Runtime cho WebXR.

IV. KIẾN TRÚC ĐỀ XUẤT

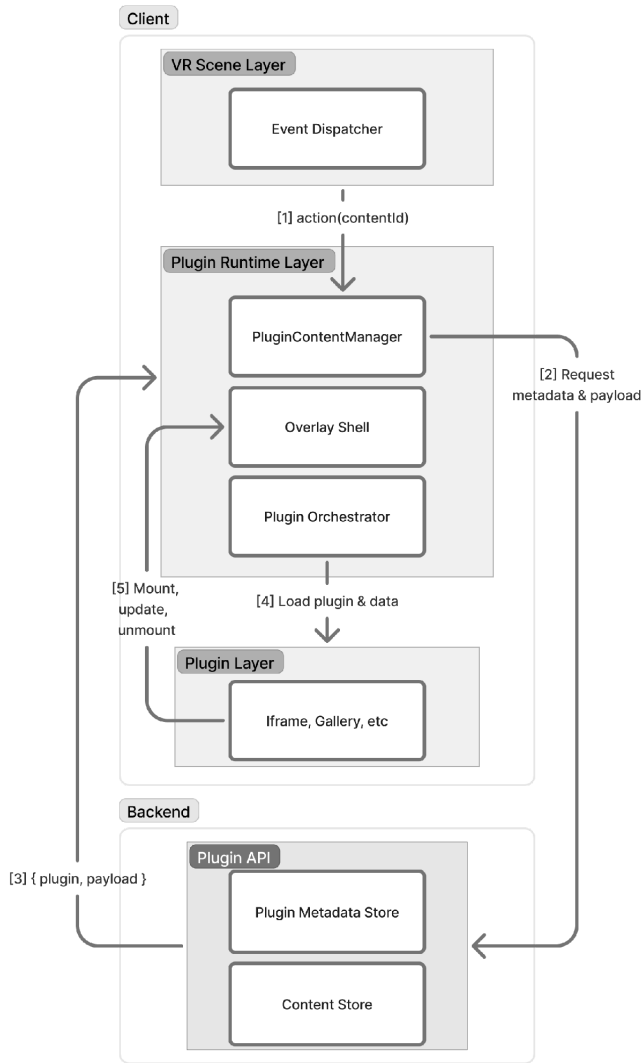
A. Kiến trúc đề xuất

Kiến trúc này được thiết kế nhằm giải quyết các hạn chế của mô hình UI nguyên khối trong WebXR, đồng thời tối ưu hóa khả năng mở rộng và tái cấu hình giao diện trong các hệ thống nhập vai.

B. Kiến trúc tổng quan

Các thành phần chính:

- Plugin Orchestrator (PluginListener): Quản lý các plugin, tải plugin, lắng nghe và xử lý các sự kiện liên quan.
- Trình tải nội dung (PluginContentManager): Là lớp trung gian nhận nội dung từ server và yêu cầu Plugin Orchestrator thực thi plugin.
- Overlay Shell: Là lớp phủ UI có thể dùng để hiển thị nội dung, điều khiển các nội dung trong VR.
- Plugin (PluginLayer): Là các plugin được viết bởi nhà phát triển.



Hình 1. Kiến trúc tổng quan

C. Bộ điều phối plugin thời gian thực

Orchestrator đảm nhiệm:

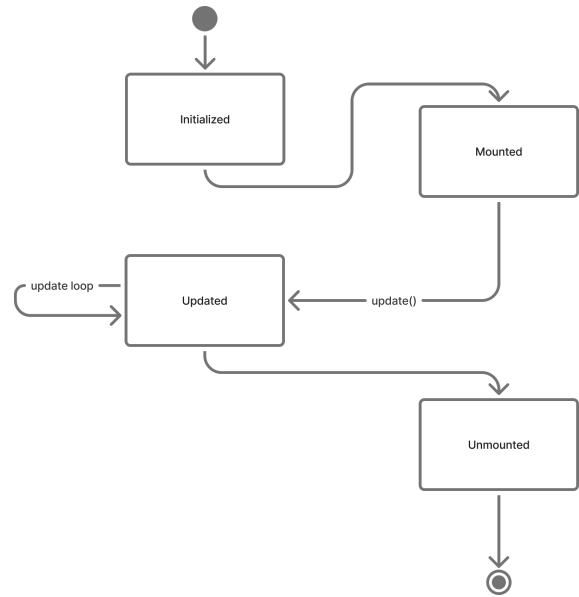
- Đăng ký plugin
- Import module thời gian thực (dynamic import)
- Tạo instance plugin
- Gọi vòng đời plugin
- Hủy plugin

Orchestrator đóng vai trò lớp điều phối trung tâm của toàn bộ micro-frontend runtime.

D. Plugin

Mỗi plugin cần phải triển khai phương thức:

- constructor(payload)
- mount(overlay)
- update(payload)
- unmount()
- updateVisibility(status)



Hình 2. Lifecycle của Plugin

Nhờ đặc tả vòng đời thống nhất, các lập trình viên có thể xây dựng nhiều loại plugin khác nhau nhưng vẫn đảm bảo tính nhất quán trong cơ chế vận hành.

E. Overlay Shell (lớp giao diện phủ)

Overlay là lớp giao diện được tách biệt khỏi môi trường VR:

- Chứa UI của plugin
- Quản lý hiển thị ẩn/hiện
- Gửi sự kiện plugins-overlay cho tất cả plugin

F. Cơ chế chọn plugin dựa trên nội dung

Backend trả về cấu trúc:

```
json
{
  "pluginName": "IFramePlugin",
  "data": { ... }
}
```

Frontend không cần biết logic, chỉ mount plugin tương ứng.

Cơ chế này cho phép backend định nghĩa hoàn toàn hành vi giao diện mà không cần sửa đổi mã nguồn frontend, từ đó tăng đáng kể khả năng tùy biến hệ thống

Trong nghiên cứu này, chúng tôi đề xuất thuật ngữ Content-defined Micro-Frontend để chỉ mô hình micro-frontend được kích hoạt và điều khiển hoàn toàn bởi cấu trúc nội dung từ backend.

G. Đánh đổi

Mô hình plugin runtime tăng chi phí network request và có thể tạo overhead bộ nhớ khi nhiều plugin hoạt động song song. Tuy nhiên, các lợi ích về tốc độ triển khai và khả năng tái cấu hình thường lớn hơn chi phí phát sinh.

V. TRIỂN KHAI HỆ THỐNG

A. Công nghệ

- Frontend VR: /A-Frame
- Plugin Shell: HTML + JS ES Modules
- Backend: Node.js + MongoDB
- Dynamic Module Loader: native dynamic import()
- Media Synchronizer: Overlay–Scene bridge

B. Case Study: IUH VR Tour với 22 địa điểm

Giải pháp áp dụng thử nghiệm cho VR Tour tại Đại học Công nghiệp TP. Hồ Chí Minh, giúp các nhà phát triển có thể tạo ra những tính năng một cách nhanh chóng, dễ dàng, đáp ứng với yêu cầu thay đổi và cập nhật liên tục của ứng dụng.

Trong quá trình thử nghiệm, các nhà phát triển đã tạo ra các tính năng tương ứng:

- Gallery Plugin: Bảng thông tin gồm nhiều hình ảnh hoặc video giới thiệu các thông tin cho các điểm chưa có hình ảnh 360.
- Iframe Plugin: Cho phép truy cập, tích hợp mini-web, điều hướng người dùng qua website mà không gián đoạn luồng người dùng.
- Notification Plugin: Hiện các thông báo như bảo trì, lỗi kỹ thuật, sự cố, v.v.
- Video Plugin: Mở popup video toàn màn hình khi tương tác một số vị trí nhất định

C. Thiết kế thực nghiệm

Các phép đo được thực hiện theo quy trình đơn giản, nhất quán:

- Mỗi kiến trúc (Monolithic UI và Plugin-based Micro-Frontend) đều chạy cùng một tính năng Overlay trên hệ thống IUH VR Tour.
- Mỗi kịch bản được chạy 10 lần, sử dụng Chrome Performance Panel, lấy giá trị trung bình quan sát được.
- Thiết bị thử nghiệm giống nhau: Acer Nitro 5, AMD Ryzen 5000 series, Chrome 134.
- Các chỉ số theo dõi: thời gian tải giao diện, thời gian chạm tương tác đầu tiên, và số lần lỗi làm vỡ toàn bộ UI.

D. Kết quả thực nghiệm

Kết quả thực nghiệm cho thấy kiến trúc Plugin-based Micro-Frontend đạt hiệu năng và mức độ linh hoạt cao hơn so với kiến trúc UI nguyên khối.

Thời gian triển khai và cập nhật tính năng giảm khoảng 67%, do mỗi plugin được build–deploy độc lập thay vì phải tải biên dịch toàn bộ giao diện.

Trong các mô phỏng lỗi, plugin cho thấy khả năng cô lập lỗi gần như hoàn toàn, tỷ lệ lỗi làm gián đoạn toàn bộ giao diện giảm đáng kể so với kiến trúc monolithic.

Chỉ số	Monolithic UI	Kiến trúc đề xuất
Thời gian tải giao diện	~526 ms	~380 ms
Độ trễ tương tác đầu tiên	~182 ms	~112 ms
Số lần lỗi gây sập UI	3/10 lần	0/10 lần
Thời gian cập nhật cho	~14 phút	~4 phút

thao tác build và deploy		
--------------------------	--	--

Mặc dù các plugin có kích thước lớn hơn, chúng được preload và cache theo phiên, dẫn tới việc thời gian tải cảm nhận (perceived load time) được rút ngắn.

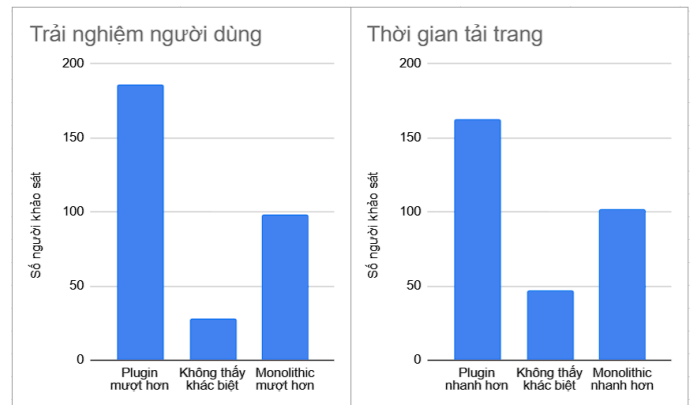
Name	Method	Status	Type	Initiator	Size	Time	Has ove...
GalleryPlugin.js	GET	200	script	PluginListener.js:65	7.0 kB	81 ms	
IframePlugin.js	GET	200	script	PluginListener.js:65	2.8 kB	90 ms	

Hình 3. JS load qua plugin, tệp nặng hơn nhưng thời gian load thấp

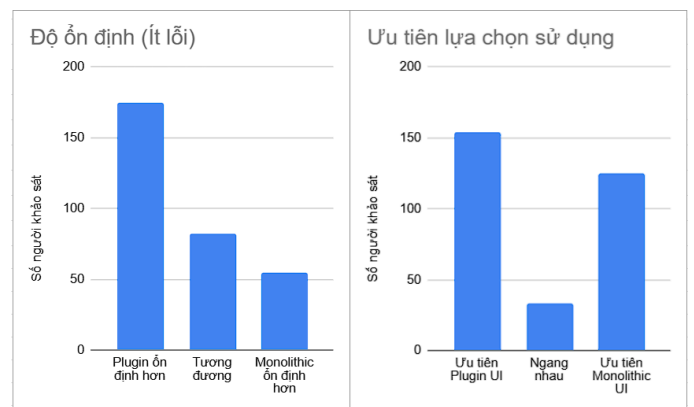
Name	Method	Status	Type	Initiator	Size	Time	Has ove...
useDataScript.js	GET	200	script	new-entry.js:39	1.8 kB	177 ms	
TourScript.js	GET	200	script	new-entry.js:40	5.0 kB	191 ms	
helper.js	GET	200	script	new-entry.js:42	1.1 kB	190 ms	
Notification.js	GET	200	script	new-entry.js:45	1.6 kB	192 ms	
api.js	GET	200	script	new-entry.js:46	1.0 kB	195 ms	
camera.js	GET	200	script	load-scene.js:2	1.1 kB	116 ms	
GroupHotspot.js	GET	200	script	load-scene.js:3	2.7 kB	116 ms	

Hình 4. JS load truyền thống, tệp nhỏ gọn nhưng load time cao hơn

Khảo sát 312 người dùng (sinh viên năm 1–3) về trải nghiệm, phản hồi UI, độ ổn định và ưu tiên lựa chọn. Kết quả ghi nhận giao diện chạy mượt hơn, phản hồi nhanh và ít giật lag khi chuyển cảnh hoặc mở overlay.



Hình 5. Đánh giá trải nghiệm và phản hồi UI



Hình 6. Đánh giá độ ổn định và ưu tiên lựa chọn

Kết quả thực nghiệm cho thấy kiến trúc mới giúp hệ thống vận hành mượt mà hơn, giảm đáng kể các lỗi giao diện và rút ngắn thời gian triển khai tính năng so với mô hình monolithic.

truyền thống. Các thao tác tải UI, chuyển cảnh và mở overlay đều diễn ra nhanh hơn và ổn định hơn.

Tổng hợp các phép đo và phản hồi người dùng cho thấy mô hình plugin mang lại lợi ích rõ ràng: hiệu năng cao hơn, lỗi ít lan truyền hơn và tốc độ cập nhật tính năng nhanh hơn đáng kể khi so sánh với kiến trúc monolithic UI.

VI. THỰC NGHIỆM VÀ ĐÁNH GIÁ

A. So sánh với kiến trúc monolithic UI

Tiêu chí	Monolithic UI	Kiến trúc đề xuất
Thêm UI mới	Build, deploy	Triển khai plugin và cho phép chúng hoạt động ngay sau khi được tải
Tái sử dụng	Thấp	Cao, có thể tái sử dụng
Cấu hình nội dung	Hard-code	Backend-defined
Overlay VR	Cần chỉnh sửa mã nguồn và build lại khi thay đổi nội dung	Mỗi plugin có thể triển khai/tắt bật độc lập, cấu hình thông qua backend
Lỗi khi tính năng chưa kịp khởi tạo	Khả năng sập cao cả hệ thống nếu bất lỗi không tốt	Không ảnh hưởng do có cơ chế fallback từ Orchestrator

VII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nghiên cứu đã giới thiệu một kiến trúc micro-frontend động dành cho WebXR, trong đó các plugin giao diện được tải và điều khiển ngay tại thời gian thực thông qua một lớp orchestrator và hệ thống overlay tách biệt khỏi cảnh VR. Cách tiếp cận này giúp giao diện không còn phụ thuộc trực tiếp vào cấu trúc scene 3D, mà có thể thay đổi linh hoạt theo nội dung backend. Nhờ vậy, trải nghiệm VR trở nên linh hoạt hơn, dễ mở rộng hơn và có khả năng thích ứng cao với nội dung. Điều này mở ra tiền đề cho các mô hình mới như “VR Experience as a Service” hoặc giao diện VR được tạo sinh (generative UI) bằng trí tuệ nhân tạo.

Tuy có nhiều ưu điểm, kiến trúc vẫn còn những hạn chế nhất định. Việc dynamic import phụ thuộc vào tốc độ mạng có thể gây chậm trễ trong quá trình tải plugin, đặc biệt trên thiết bị di động hoặc mạng yếu. Nếu plugin bao gồm nhiều thành phần đồ họa hoặc tác vụ xử lý phức tạp, hệ thống có thể gặp suy giảm tốc độ khung hình. Ngoài ra, kiến trúc chưa được thử nghiệm trên các headset VR chuyên dụng (Meta Quest, Pico), nơi môi trường WebXR và yêu cầu hiệu năng sẽ khắt khe hơn. Một vấn đề quan trọng khác là cơ chế kiểm duyệt và bảo mật plugin vẫn chưa hoàn thiện, tạo ra rủi ro khi hệ thống mở cho nhiều bên phát triển plugin.

Trong thời gian tới, có thể mở rộng nghiên cứu theo một số hướng sau. Thứ nhất, tối ưu hóa tốc độ tải plugin thông qua preloading, caching hoặc sử dụng hạ tầng CDN để giảm trễ mạng. Thứ hai, xây dựng cơ chế bảo mật chuyên biệt cho plugin, chẳng hạn như sandbox kiểm tra mã hoặc bộ quy tắc kiểm duyệt trước khi plugin được phép chạy. Thứ ba, cần tiến hành thử nghiệm và tối ưu cho các headset VR chuyên dụng nhằm đánh giá mức độ tương thích và hiệu năng thực tế. Cuối

cùng, việc tích hợp trí tuệ nhân tạo để gợi ý, chọn hoặc tạo plugin dựa trên hành vi người dùng sẽ giúp hệ thống trở nên thông minh và cá nhân hóa hơn. Những hướng phát triển này sẽ góp phần hoàn thiện kiến trúc và mở rộng phạm vi ứng dụng trong giáo dục, triển lãm số, du lịch ảo và nhiều lĩnh vực tương tác 3D khác.

Tổng thể, nghiên cứu đóng góp một hướng kiến trúc khả thi cho việc hiện thực hóa micro-frontend trong môi trường WebXR — lĩnh vực vốn còn thiếu các giải pháp tiêu chuẩn hóa.

REFERENCES

- [1] Mozilla, “WebXR Device API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API. Accessed: Nov. 20, 2025.
- [2] A-Frame, “A-Frame: Web framework for building virtual reality experiences.” [Online]. Available: <https://aframe.io>. Accessed: Nov. 20, 2025.
- [3] D. S. Ceulemans, S. Jacobs, E. Van de Weghe and P. De Maeyer, “360-degree virtual tour for educational purposes: An exploration on the design considerations and decisions,” in Proc. 14th Int. CDIO Conf., Kanazawa, Japan, 2018.
- [4] H. Cardona, J. P. Fernández and P. C. González, “Virtual tours to facilities for educational purposes: A review,” TEM Journal, vol. 12, no. 3, pp. 1725–1731, 2023, doi: 10.18421/TEM123-55.
- [5] S. Zhao, S. Yang and D. W. Shaffer, “Learning in the field: Comparison of desktop, immersive virtual reality and actual field trips for place-based STEM education,” in 2020 IEEE Conf. Virtual Reality and 3D User Interfaces (VR), pp. 893–902, 2020, doi: 10.1109/VR46266.2020.00012.
- [6] L. Mezzalana, Building Micro-Frontends: Scaling Teams and Projects, Empowering Developers. Sebastopol, CA: O’Reilly Media, 2021.
- [7] C. Jackson, “Micro Frontends,” martinfowler.com, 2019. [Online]. Available: <https://martinfowler.com/articles/micro-frontends.html>. Accessed: Nov. 20, 2025.
- [8] Webpack, “Module Federation,” webpack.js.org, 2024. [Online]. Available: <https://webpack.js.org/concepts/module-federation/>. Accessed: Nov. 20, 2025.
- [9] J. Radianti, T. A. Majchrzak, J. Fromm and I. Wohlgenannt, “A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned and research agenda,” Computers & Education, vol. 147, Art. no. 103778, 2020, doi: 10.1016/j.compedu.2019.103778.
- [10] Google Developers, “Web Components,” Web Fundamentals. [Online]. Available: <https://developers.google.com/web/fundamentals/web-components/>. Accessed: Nov. 20, 2025.
- [11] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.
- [12] W3C, “WebXR Device API – Editor’s Draft.” [Online]. Available: <https://www.w3.org/TR/webxr/>. Accessed: Nov. 20, 2025.
- [13] G. Makransky and L. Lilleholt, “A structural equation modeling investigation of the emotional value of immersive virtual reality in education,” Educational Technology Research and Development, vol. 66, no. 5, pp. 1141–1164, 2018, doi: 10.1007/s11423-018-9581-2.
- [14] S. Damiani, M. Pirker and G. Fitzpatrick, “Adaptive and intelligent interfaces in virtual reality: A systematic review,” in Proc. ACM CHI Workshop on Adaptive Immersive Environments, 2022.